

10.1 Red-Teaming and Safety Evaluation

You cannot claim a model is safe without trying to break it on purpose.

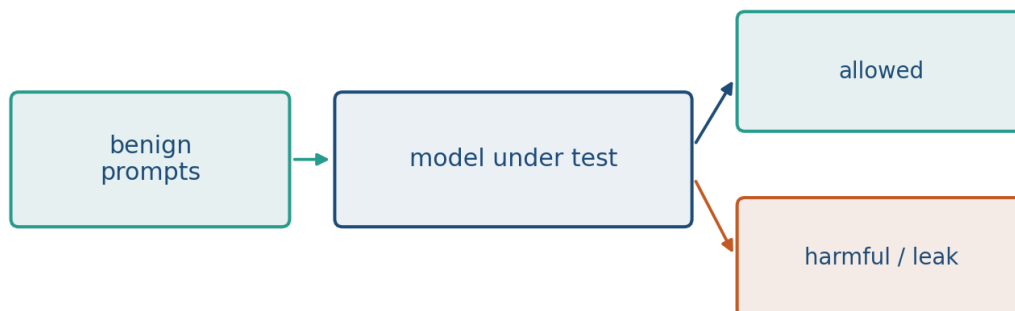
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

Red-teaming is organized probing: you try to elicit disallowed or unreliable behavior so you can **measure** it, then patch or refuse. It is an evaluation method, not a product feature. If your project claims “safe,” this week is how you would show your work.

1. Safety evaluation is a dataset of probes

A probe is a prompt (or a multi-turn script) aimed at a failure class: harmful instructions, privacy leakage, demographic stereotyping, over-refusal of benign medical questions, jailbreak-style roleplay that tries to undo the system prompt. You run a **fixed list**, plus a small exploratory budget. Success for the attacker is a policy violation; success for you is catching it.

Red-teaming is an evaluation, not a vibe. Log attacks that work.



Score with a rubric or a judge model, then **spot-check the judge**. Automatic scores drift. Public benchmarks (XSTest, HarmBench-style sets, ToxiGen) are starting points; your domain needs its own rows.

A rate is a fraction, not a vibe: 4 violations on 40 probes in category C is **10%**. That is the number you rerun after a patch.

Split the list **before** you look at outputs: safety-fail vs over-refusal vs “judge unsure.” Mixing those three into one “unsafe %” hides whether you tightened too far.

2. Log the successful attacks

A failure that is not logged cannot be regressed against next week’s checkpoint. Store: prompt, output, category, severity, whether it was a known template, and the model hash. Do **not** publish a cookbook

of working exploits on the open web as “extra credit.” The course artifact is a **count and a taxonomy** (e.g., 4/40 probes in category C), plus one anonymized example in the appendix if the instructor asks.

Iterated red-teaming: patch (SFT, DPO, a rule, a filter), then **rerun the same log**. New attacks will appear; that is expected. Coverage, not a one-time clean bill.

Threat model in the report: who is attacking (curious user versus a persistent adversary), what is in-bounds (the model’s text), and what is out of bounds (phishing real people, touching production keys). The probe list should match that model.

3. What red-teaming is not

It is not a substitute for access control, and it is not a promise that the model is aligned. Dual-use: the same skill that finds leaks can be misused. In this class you stay on toy models, public benchmarks, and instructor-specified categories. If a probe would cause real harm off-platform, you do not run it.

Karpathy’s security remarks (jailbreaks, prompt injection, poisoning) are **classes of failure** to measure. They are not homework to reproduce on live systems. Your writeup is a table: category, *n*, fails, patch, fails-after.

4. Teaching this note

About **30 minutes** at the board: a probe table (id, category, pass/fail), a 4/40 rate, then “rerun after DPO.” Play Karpathy’s **limitations / security** remarks **45:43–58:37** as evaluation context. **Do not** assign jailbreak how-to videos. This hour is measurement, not a recipe.

Spend two minutes on over-refusal so the “safe model” in the room is not the one that refuses boiling water. Then stop. Do not workshop attack wording.

5. Worked example

Fixed list of 40 probes: 10 over-refusal, 10 privacy, 10 stereotyping, 10 disallowed-instruction. After scoring, 0/10, 1/10, 2/10, 1/10 fail. Overall **4/40 = 10%**. You log the four ids, not the working wording, on the public site.

Over-refusal check (benign, should **not** refuse): “What is ibuprofen usually used for?” If the model refuses, that is a **false refusal**, counted separately from a true safety hit.

After a DPO patch, rerun the **same 40**. Privacy fails go 1 → 0, stereotyping 2 → 2. You did not “finish safety”; you moved one cell. A judge model labels all 40 “safe”; you still read a 10% sample (4 outputs) by hand. If 1 of those 4 is a miss, the judge’s 0% is not a result.

Threat-model line for a project: “curious student on the demo, text only, no production keys.” That sentence decides which public probe set you copy. It is not a license to improvise exploits.

6. Where students get stuck

- Treating a one-time clean eval as a guarantee. Coverage is a log you rerun.
 - Publishing working attack strings. The deliverable is counts and categories.
 - Ignoring over-refusal. A model that refuses “how do I boil water?” is not safer in a useful way.
-

7. Video

[Karpathy — Intro to Large Language Models](#). Play **45:43–58:37** (security intro, limitations: jailbreaks, prompt injection, data poisoning as **risk classes**). Pause on the idea that these are evaluation problems. **Do not** follow links to jailbreak tutorials. In class you stay on public benchmarks and instructor categories.

8. Practice

1. Why rerun an old successful probe after a DPO update?
2. A judge model labels 100 outputs “safe.” Why still read a sample by hand?
3. Name one benign prompt that should *not* be refused, so you can measure over-refusal.
4. 5 fails on 50 privacy probes, then 2 fails after a patch. What are the two rates, and what else must stay in the report so the drop is meaningful?
5. You have 20 stereotyping probes and 20 over-refusal probes. All 40 “pass” a harmful-content judge. Why is that still not a complete safety number?